Note to other teachers and users of these slides: We would be delighted if you found our material useful for giving your own lectures. Feel free to use these slides verbatim, or to modify them to fit your own needs. If you make use of a significant portion of these slides in your own lecture, please include this message, or a link to our web site: http://cs224w.Stanford.edu

Stanford CS224W: GNNs for Recommender Systems

CS224W: Machine Learning with Graphs
Jure Leskovec, Stanford University
http://cs224w.stanford.edu



Announcements

- **Homework 2** is due Thursday 10/29
- **Homework 3** will be out on Thursday 10/29
  - Recitation session details TBA on Ed
- **No class next Tuesday 11/3 (Democracy Day)**
- **Colab 3** is due next Thursday 11/5
- **Project Milestone** is due Tuesday 11/10
  - 2% of overall course grade
  - Graded on credit/no credit basis

Stanford CS224W: Recommender Systems: Task and Evaluation

CS224W: Machine Learning with Graphs

Jure Leskovec, Stanford University

http://cs224w.stanford.edu

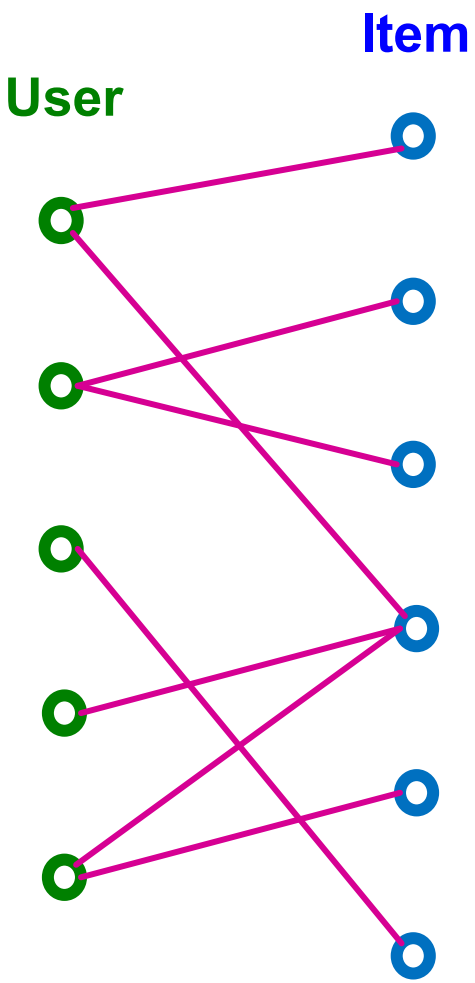


Preliminary of Recommendation

- **Information Explosion in the era of Internet**
  - 10K+ movies in Netflix
  - 12M products in Amazon (350m on Marketplace)
  - 70M+ music tracks in Spotify
  - 10B+ videos on YouTube
  - 200B+ pins (images) in Pinterest
- **Personalized recommendation (i.e., suggesting a small number of interesting items for each user)** is critical for users to effectively explore the content of their interest.

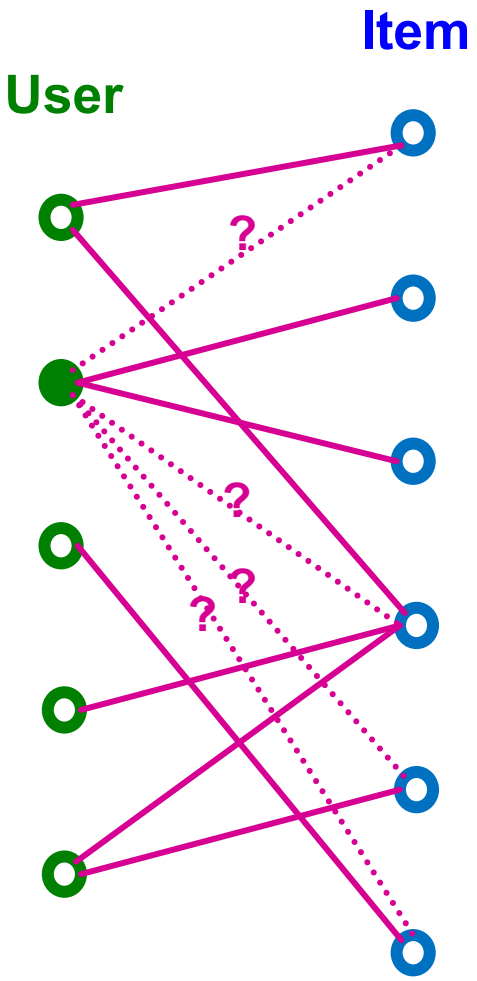
Recommender System as a Graph

- Recommender system can be naturally modeled as a **bipartite graph**
  - A graph with two node types: **users** and **items**.
  - **Edges** connect users and items
    - Indicates user-item interaction (e.g., click, purchase, review etc.)
    - Often associated with timestamp (timing of the interaction).



Recommendation Task

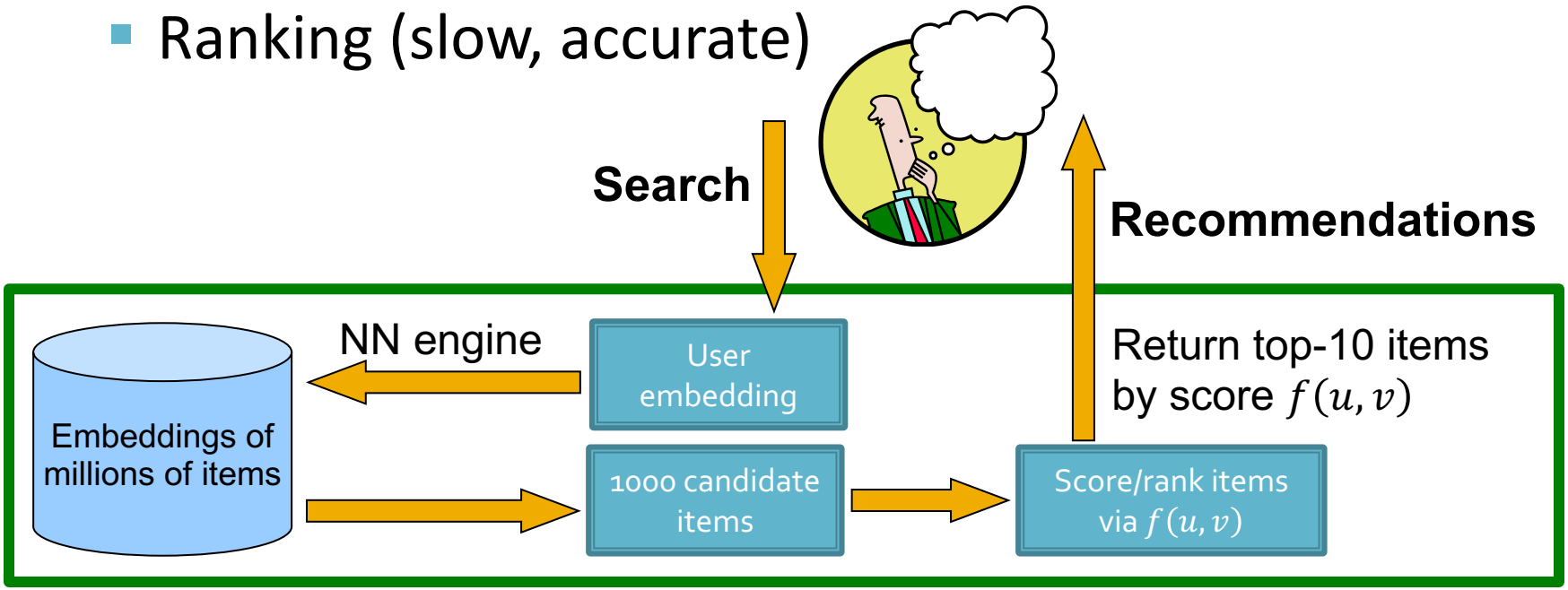
- **Given**
  - Past user-item interactions
- **Task**
  - Predict new items each user will interact in the future.
  - Can be cast as **link prediction** problem.
    - Predict new user-item interaction edges given the past edges.
  - For $u \in \boldsymbol{U}, v \in \boldsymbol{V}$, we need to get a real-valued **score** $f(u, v)$.



Modern Recommender System

- **Problem:** Cannot evaluate $f(u, v)$ for every user u – item v pair.
- **Solution:** 2-stage process:
  - Candidate generation (cheap, fast)
  - Ranking (slow, accurate)

Example $f(u, v)$:
$f(u, v) = z_u \cdot z_v$

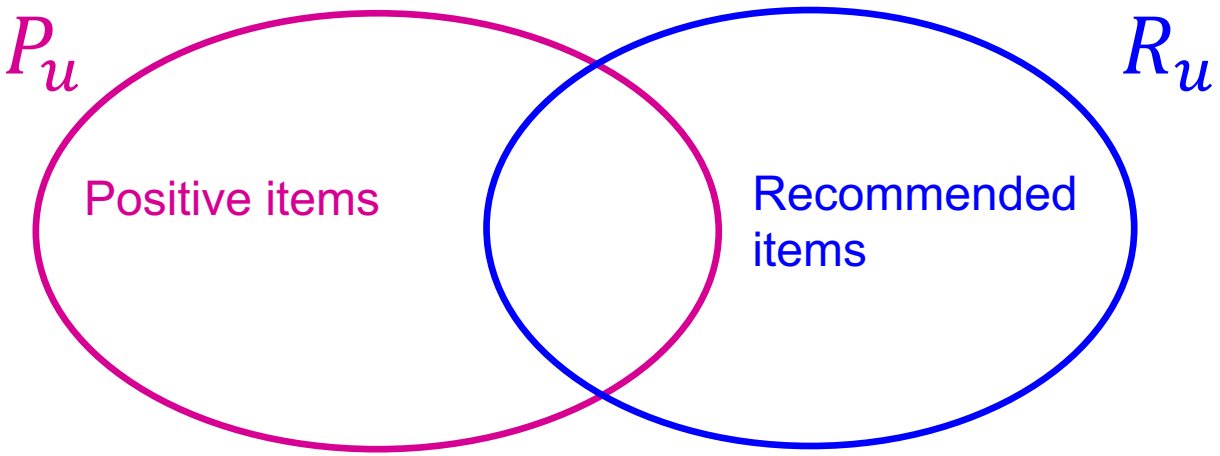


Top-K Recommendation

- For each user, we recommend K items.
  - **For recommendation to be effective, K needs to be much smaller than the total number of items (up to billions)**
  - K is typically in the order of 10—100.
- The goal is to include as many **positive items** as possible in the top-K recommended items.
  - **Positive items = Items that the user will interact with in the future.**
- **Evaluation metric**: Recall@K (defined next)

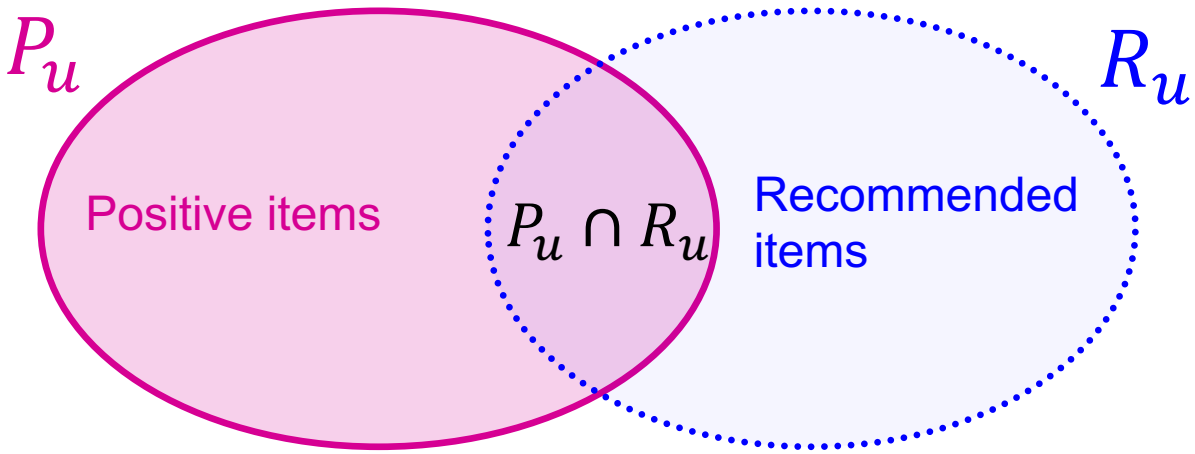
Evaluation Metric: Recall@K (1)

- **For each user u,**
  - Let P_u be a set of positive items the user will interact in the future.
  - Let R_u be a set of items recommended by the model.
    - In top-K recommendation, $|R_u| = K$.
    - Items that the user has already interacted are excluded.



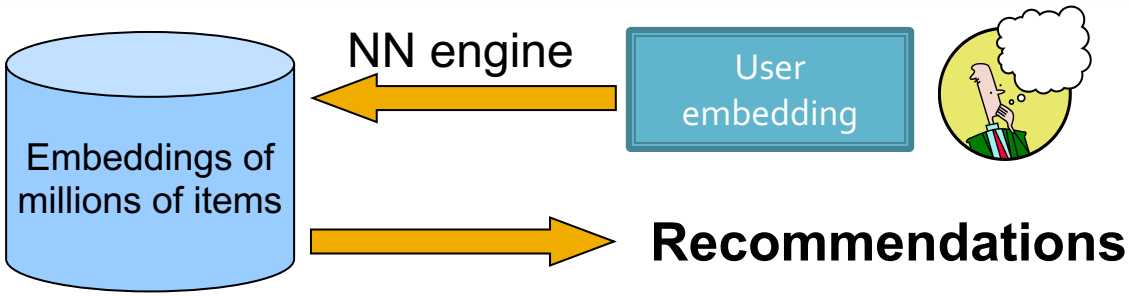
Evaluation Metric: Recall@K (2)

- **Recall@K for user u is $|P_u \cap R_u|/|P_u|$.**
  - Higher value indicates more positive items are recommended in top-K for user u.



- The final Recall@K is computed by averaging the recall values across all users.

Recommendations with LLMs?



- **GNN** to embed users & products
- **LLM** to textify products and embed them
  - user emb. is avg. of purchased product embs.
- H&M fashion recommendation:

	MAP@12	PRECISION@12	RECALL@12	F1@12
LLM	0.00190 (-93.33%)	0.00329 (-67.84%)	0.00119 (-97.73%)	0.0071 (-54.60%)
GNN	0.02856 15x better	0.01023 3x	0.05234 40x	0.01564 2x

More info: [Do Large Language Models make accurate personalized recommendations?]

Stanford CS224W: Recommender Systems: Embedding-Based Models

CS224W: Machine Learning with Graphs
Jure Leskovec, Stanford University
http://cs224w.stanford.edu

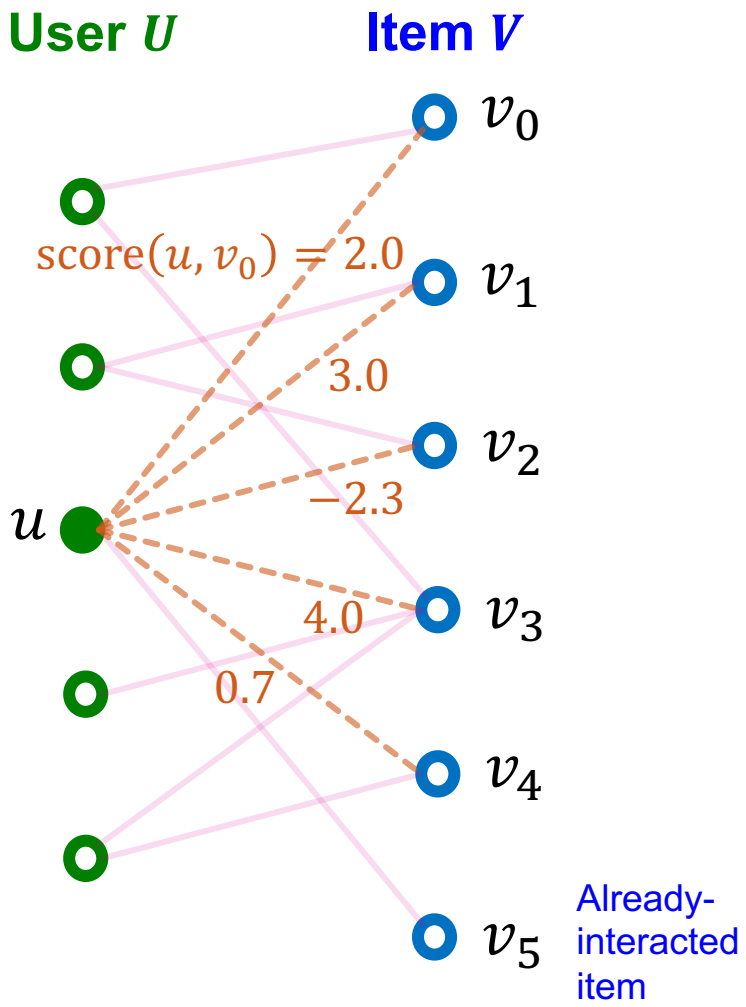


Notation

- **Notation**:
  - $\boldsymbol{U}$: A set of all users
  - $\boldsymbol{V}$: A set of all items
  - $\boldsymbol{E}$: A set of observed user-item interactions
    - $\boldsymbol{E} = \{(u, v) \mid u \in \boldsymbol{U}, v \in \boldsymbol{V}, u \text{ interacted with } v\}$

Scoring Function

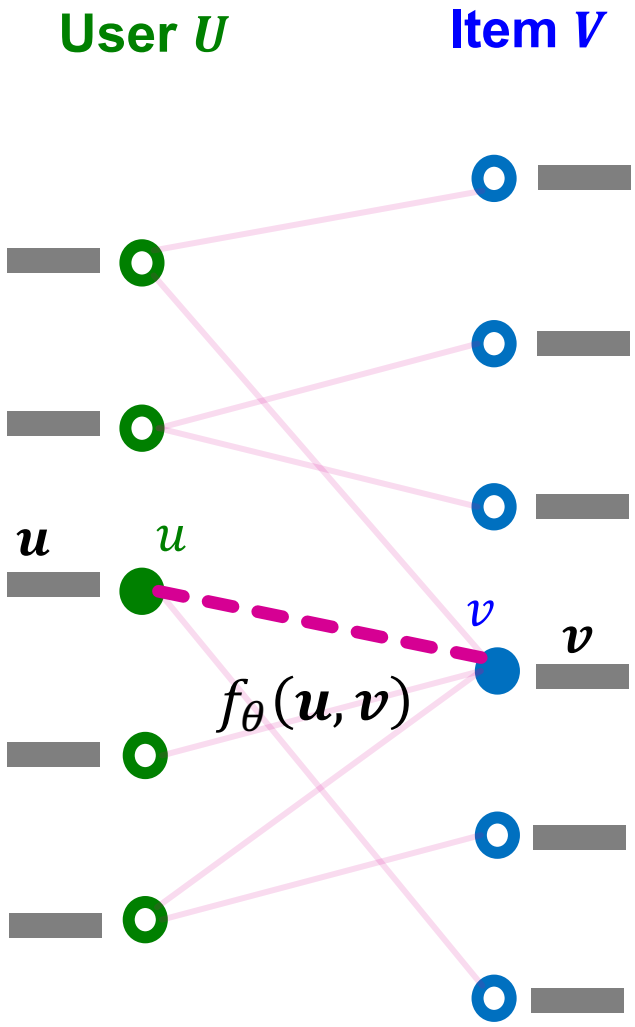
- To get the top-K items, we need a score function for user-item interaction:
  - For $u \in U, v \in V$, we need to get a real-valued scalar **score**(u, v).
  - ***K* items with the largest scores for a given user u** (excluding already-interacted items) are then recommended.



For $K = 2$, recommended items for user u would be $\{v_1, v_3\}$.

Embedding-Based Models

- We consider **embedding-based models** for scoring user-item interactions.
  - For each user $u \in \boldsymbol{U}$, let $\boldsymbol{u} \in \mathbb{R}^D$ be its D-dimensional embedding.
  - For each item $v \in \boldsymbol{V}$, let $\boldsymbol{v} \in \mathbb{R}^D$ be its D-dimensional embedding.
  - Let $f_\theta(\cdot,\cdot): \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$ be a parametrized function.
  - Then, $\text{score}(u, v) \equiv f_\theta(\boldsymbol{u}, \boldsymbol{v})$



Training Objective

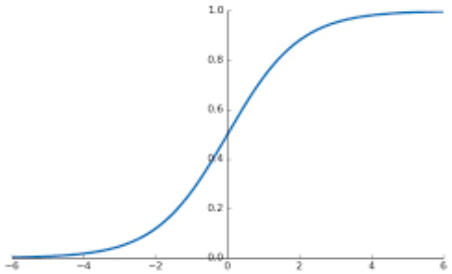
- Embedding-based models have three kinds of parameters:
  - An encoder to generate user embeddings $\{\boldsymbol{u}\}_{u \in U}$
  - An encoder to generate item embeddings $\{\boldsymbol{v}\}_{v \in V}$
  - Score function $f_\theta(\cdot,\cdot)$
- **Training objective**: Optimize the model parameters to achieve high recall@K on *seen* (i.e., *training*) user-item interactions
  - We hope this objective would lead to high recall@K on *unseen* (i.e., *test*) interactions.

Surrogate Loss Functions

- The original training objective (recall@K) is ***not* differentiable**.
  - *Cannot* apply efficient gradient-based optimization.
- Two **surrogate loss functions** are widely-used to enable efficient gradient-based optimization.
  - Binary loss
  - Bayesian Personalized Ranking (BPR) loss
- Surrogate losses are **differentiable** and should **align well with the original training objective**.

Binary Loss (1)

- Define **positive/negative edges**
  - A set of **positive edges** $\boldsymbol{E}$ (i.e., observed/training user-item interactions)
  - A set of **negative edges** $\boldsymbol{E_{\text{neg}}} = \{(u, v) \mid (u, v) \notin E, u \in \boldsymbol{U}, v \in \boldsymbol{V}\}$
- Define **sigmoid function** $\sigma(x) \equiv \frac{1}{1+\exp(-x)}$
  - Maps real-valued scores into binary likelihood scores, i.e., in the range of [0,1].



Binary Loss (2)

- **Binary loss**: Binary classification of **positive**/**negative** edges using $\sigma(f_\theta(\boldsymbol{u}, \boldsymbol{v}))$:

$$\underbrace{-\frac{1}{|\boldsymbol{E}|}\sum_{(u,v)\in \boldsymbol{E}} \log\left(\sigma(f_\theta(\boldsymbol{u}, \boldsymbol{v}))\right)}_{} \underbrace{-\frac{1}{|\boldsymbol{E}_{\mathbf{neg}}|}\sum_{(u,v)\in \boldsymbol{E}_{\mathbf{neg}}} \log\left(1-\sigma(f_\theta(\boldsymbol{u}, \boldsymbol{v}))\right)}_{}$$

During training, these terms can be approximated using mini-batch of positive/negative edges

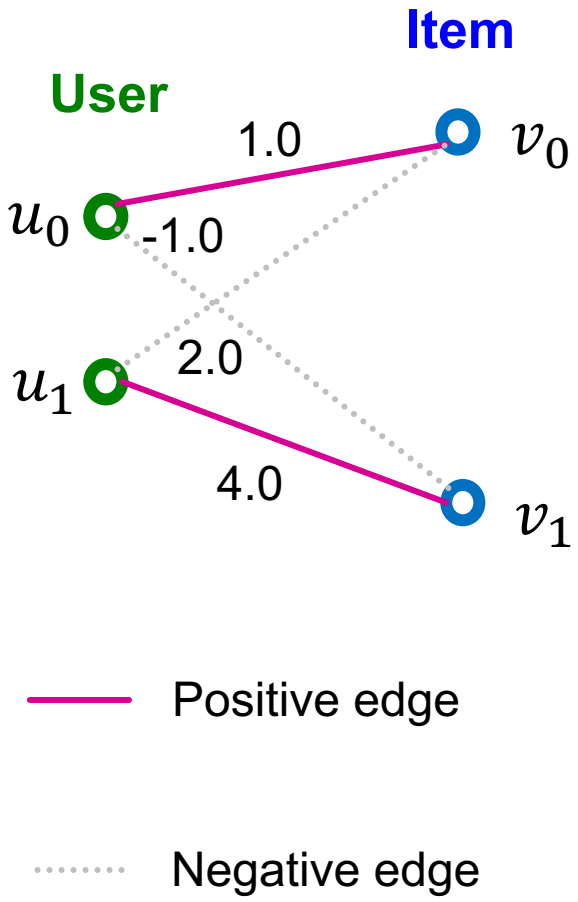
- Binary loss pushes the scores of **positive edges** higher than those of **negative edges**.
  - This aligns with the training recall metric since positive edges need to be recalled.

Issue with Binary Loss (1)

- **Issue**: In the binary loss, the scores of ***ALL*** positive edges are pushed higher than those of ***ALL*** negative edges.
- This would unnecessarily penalize model predictions even if the training recall metric is perfect.
- **Why?** (example in the next slide)

Issue with Binary Loss (2)

- **Let's consider the simplest case:**
  - Two users, two items
  - Metric: Recall@1.
  - A model assigns the score for every user-item pair (as shown in the right).
- Training **Recall@1 is 1.0** (perfect score), because v_0 (resp. v_1) is correctly recommended to u_0 (resp. u_1).
- However, **the binary loss would still penalize the model prediction** because the negative (u_1, v_0) edge gets the higher score than the positive edge (u_0, v_0).



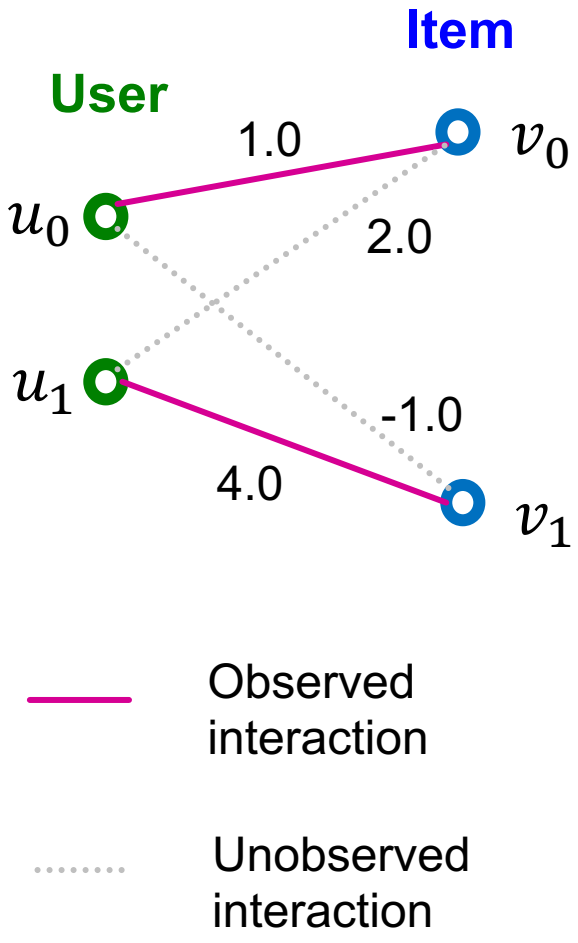
Issue with Binary Loss (3)

- **Key insight**: The binary loss is **non-personalized** in the sense that the **positive/negative edges are considered *across ALL users at once*.**

- However, the recall metric is inherently **personalized (defined for each user)**.
  - The non-personalized binary loss is overly-stringent for the personalized recall metric.

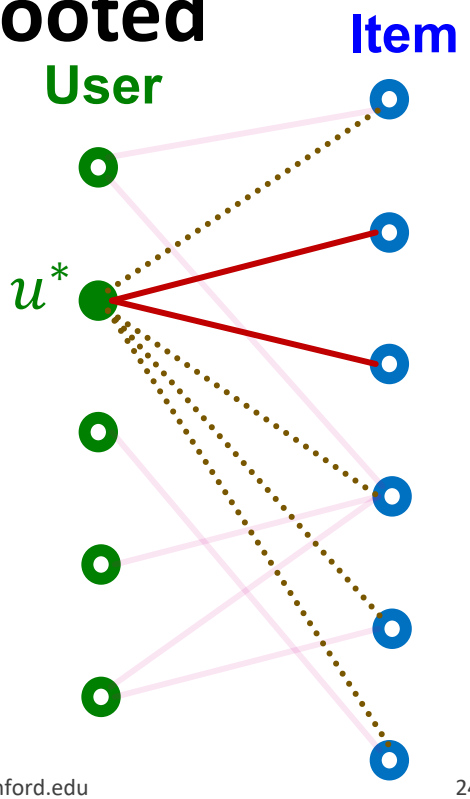
Desirable Surrogate Loss

- **Lesson learned**: Surrogate loss function should be defined in a **personalized** manner.
  - **For each user**, we want the scores of positive items to be higher than those of the negative items
  - We do *not* care about the score ordering across users.
- **Bayesian Personalized Ranking (BPR)** loss achieves this!



Loss Function: BPR Loss (1)

- **Bayesian Personalized Ranking (BPR) loss** is a personalized surrogate loss that aligns better with the recall@K metric.
- For each user $u^* \in \boldsymbol{U}$, define the **rooted positive/negative edges** as
  - Positive edges rooted at u^*
    - $\boldsymbol{E}(u^*) \equiv \{(u^*, v) \mid (u^*, v) \in \boldsymbol{E}\}$
  - Negative edges rooted at u^*
    - $\boldsymbol{E}_{\mathbf{neg}}(u^*) \equiv \{(u^*, v) \mid (u^*, v) \in \boldsymbol{E}_{\mathbf{neg}}\}$



Note: The term "Bayesian" is not essential to the loss definition. The original paper [Rendle et al. 2009] considers the Bayesian prior over parameters (essentially acts as a parameter regularization), which we omit here.

Loss Function: BPR Loss (2)

- **Training objective**: **For each user** u^*, we want the scores of rooted positive edges $\boldsymbol{E}(u^*)$ to be higher than those of rooted negative edges $\boldsymbol{E}_{\mathbf{neg}}(u^*)$.
  - **Aligns with the personalized nature of the recall metric.**
- **BPR Loss for user u^*:**

Encouraged to be positive for each user
=positive edge score is higher than negative edge score

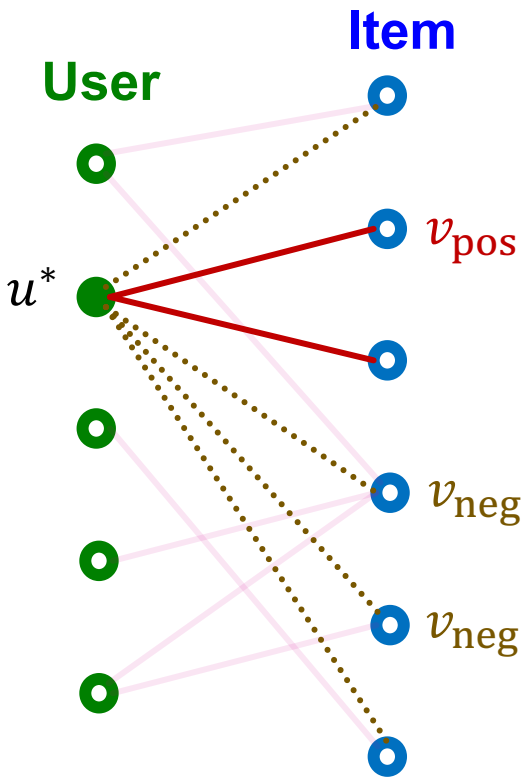
$$\text{Loss}(u^*) = \frac{1}{|E(u^*)| \cdot |\boldsymbol{E}_{\mathbf{neg}}(u^*)|} \underbrace{\sum_{(u^*, v_{\text{pos}}) \in \boldsymbol{E}(u^*)} \sum_{(u^*, v_{\text{neg}}) \in \boldsymbol{E}_{\mathbf{neg}}(u^*)} -\log\left(\sigma\left(f_\theta(\boldsymbol{u}^*, \boldsymbol{v}_{\text{pos}}) - f_\theta(\boldsymbol{u}^*, \boldsymbol{v}_{\text{neg}})\right)\right)}$$

Can be approximated using a mini-batch

- Final BPR Loss: $\frac{1}{|\boldsymbol{U}|} \sum_{u^* \in \boldsymbol{U}} \text{Loss}(u^*)$

Loss Function: BPR Loss (3)

- **Mini-batch training for the BPR loss:**
  - In each mini-batch, we sample a subset of users $\boldsymbol{U}_{\text{mini}} \subset \boldsymbol{U}$.
    - For each user $u^* \in \boldsymbol{U}_{\text{mini}}$, we sample one positive item v_{pos} and a set of sampled negative items $V_{\text{neg}} = \{v_{\text{neg}}\}$.
  - The mini-batch loss is computed as



$$\frac{1}{|\boldsymbol{U}_{\text{mini}}|} \sum_{u^* \in \boldsymbol{U}_{\text{mini}}} \frac{1}{|\boldsymbol{V}_{\text{neg}}|} \sum_{v_{\text{neg}} \in \boldsymbol{V}_{\text{neg}}} -\log\left(\sigma\left(f_\theta(u^*, \boldsymbol{v}_{\text{pos}}) - f_\theta(u^*, \boldsymbol{v}_{\text{neg}})\right)\right)$$

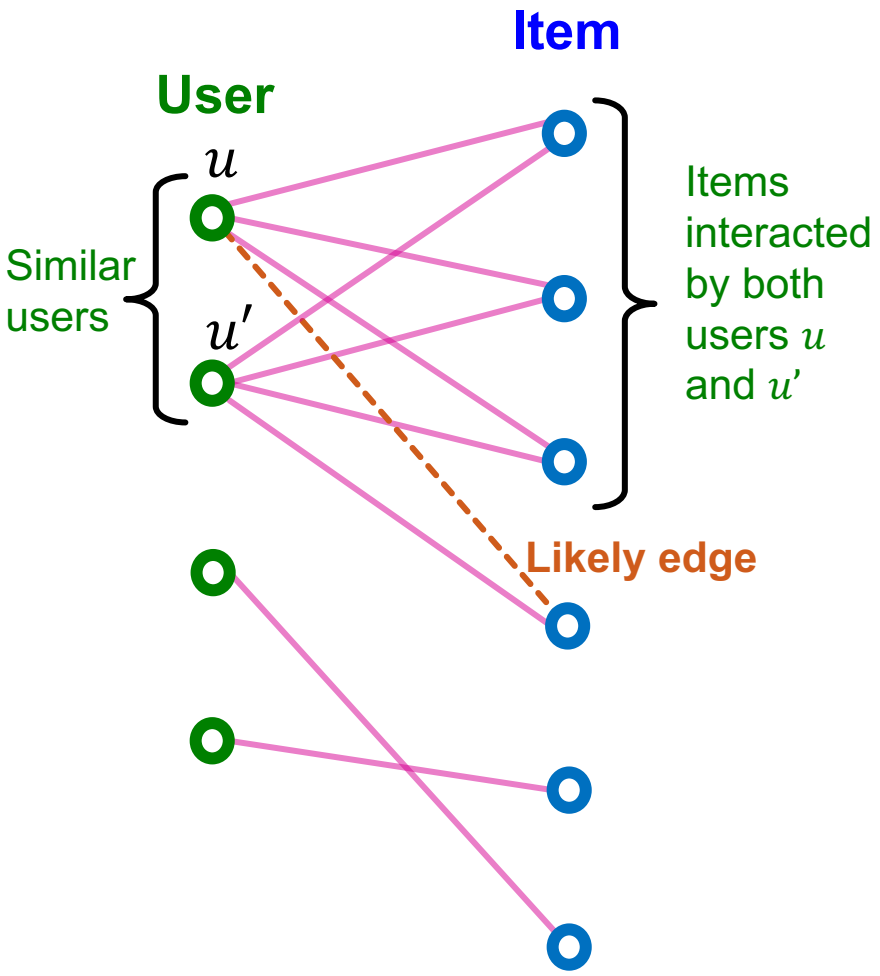
Average over users in the mini-batch

Summary So Far

- **We have introduced**
  - Recall@K as a metric for personalized recommendation
  - Embedding-based models
    - Three kinds of parameters to learn
      - **user encoder** to generate user embeddings
      - **item encoder** to generate item embeddings
      - **score function** to predict the user-item interaction likelihood.
  - Surrogate loss functions to achieve the high recall metric.
- Embedding-based models have achieved SoTA in recommender systems.
  - **Why do they work so well?**

Why Embedding Models Work?

- **Underlying idea: Collaborative filtering**
  - Recommend items for a user by **collecting preferences of many other similar users.**
  - **Similar users tend to prefer similar items.**
- **Key question: How to capture similarity between users/items?**



Why Embedding Models Work?

- Embedding-based models can capture similarity of users/items!
  - **Low-dimensional embeddings *cannot* simply memorize all user-item interaction data.**
  - Embeddings are forced to **capture similarity between users/items to fit the data.**
  - This allows the models to make effective prediction on *unseen* user-item interactions.

This Lecture: GNNs for Recsys

- **In this lecture, we teach two representative GNN approaches for recommender systems.**
- **(1) Neural Graph Collab. Filtering (NGCF)** [Wang et al. 2019]
- **(2) LightGCN** [He et al. 2020]
  - Improve the conventional collaborative filtering models (i.e., shallow encoders) by explicitly modeling graph structure using GNNs.
  - Assumes no user/item features.
- **PinSAGE** [Ying et al. 2018]
  - Use GNNs to generate high-quality embeddings by simultaneously capturing rich node attributes (e.g., images) and the graph structure.

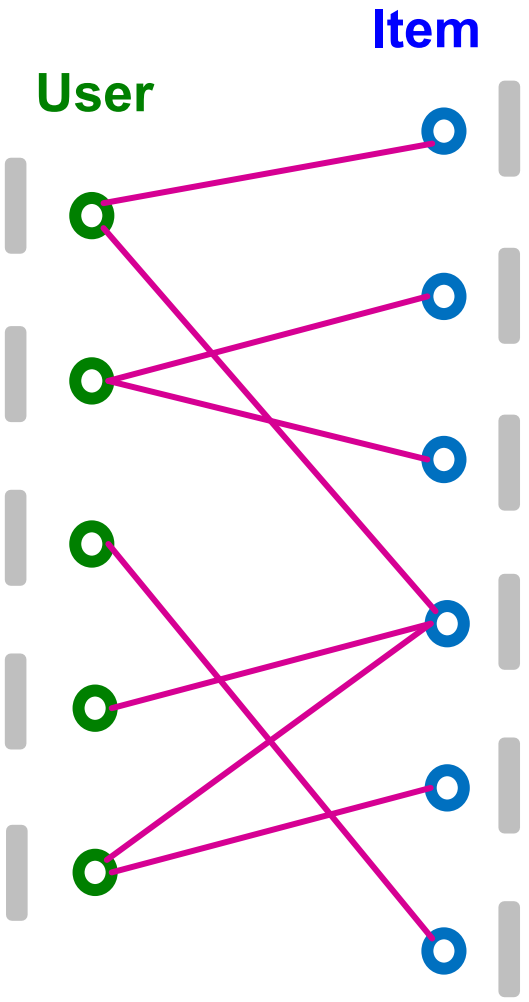
Stanford CS224W: Neural Graph Collaborative Filtering

CS224W: Machine Learning with Graphs
Jure Leskovec, Stanford University
http://cs224w.stanford.edu



Conventional Collaborative Filtering

- Conventional collaborative filtering model is based on **shallow encoders**:
  - No user/item features.
  - Use shallow encoders for users and items:
    - For every $u \in U$ and $v \in V$, we prepare shallow learnable embeddings $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^D$.
  - Score function for user u and item v is $f(\boldsymbol{u}, \boldsymbol{v}) \equiv \boldsymbol{z_u}^T \boldsymbol{z_v}$.



Learnable shallow user/item embeddings

Limitations of Shallow Encoders

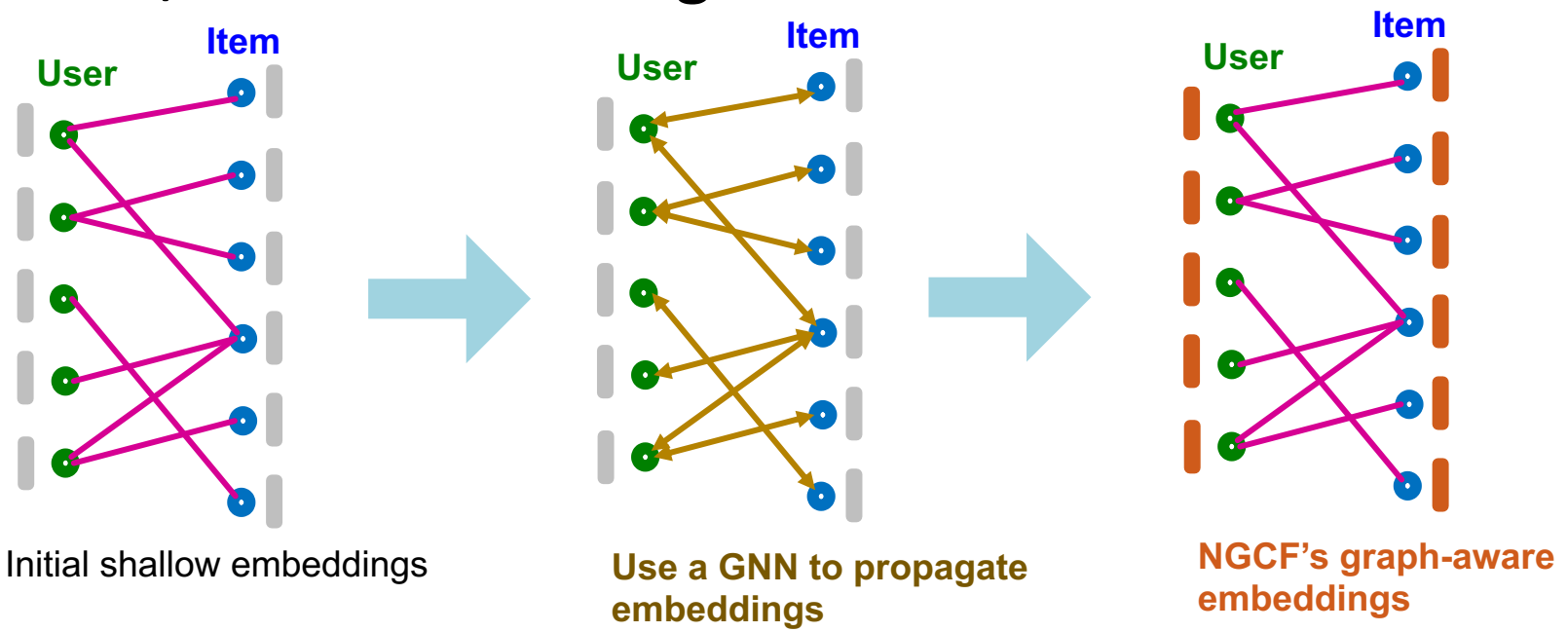
- The model itself does ***not explicitly*** capture graph structure
  - The graph structure is ***only implicitly*** captured in the training objective.
- Only the **first-order graph structure** (i.e., edges) is captured in the training objective.
  - **High-order graph structure** (e.g., K-hop paths between two nodes) is ***not*** **explicitly captured.**

Motivation

- We want a model that…
  - **explicitly captures graph structure** (beyond implicitly through the training objective)
  - captures **high-order graph structure** (beyond the first-order edge connectivity structure)
- **GNNs are a natural approach to achieve both!**
  - **Neural Graph Collaborative Filtering** (NGCF) [Wang et al. 2019]
  - **LightGCN** [He et al. 2020]
    - A simplified and improved version of NGCF

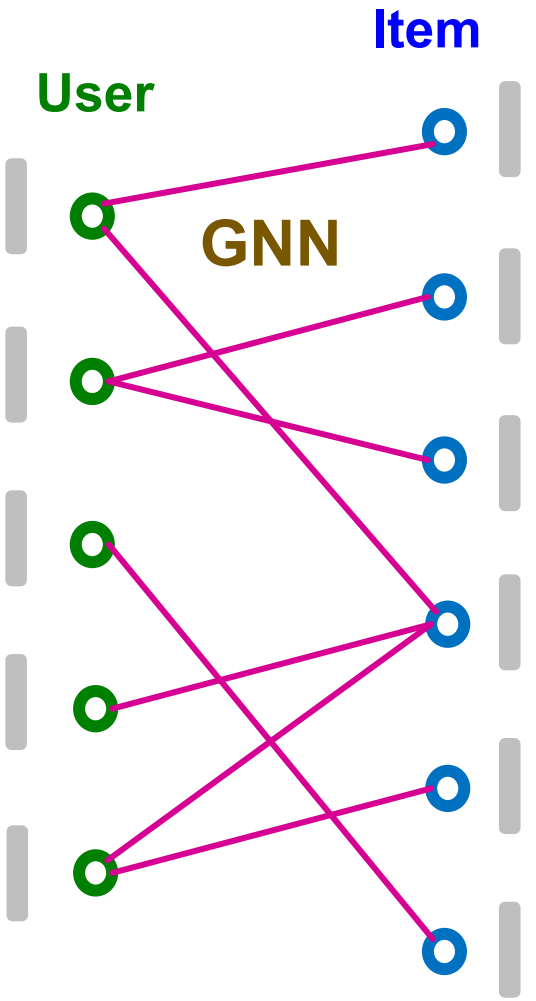
NGCF: Overview

- **Neural Graph Collaborative Filtering (NGCF)** ***explicitly*** incorporates high-order graph structure when generating user/item embeddings.
- **Key idea**: Use a GNN to generate graph-aware user/item embeddings.



NGCF Framework

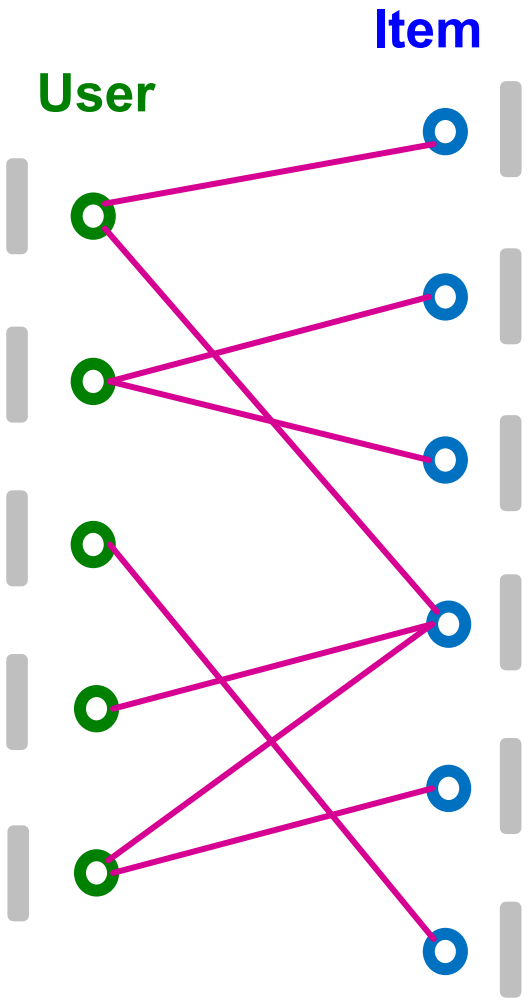
- **Given**: User-item bipartite graph.
- **NGCF framework:**
  - Prepare shallow learnable embedding for each node.
  - Use multi-layer GNNs to propagate embeddings along the bipartite graph.
    - High-order graph structure is captured.
  - Final embeddings are *explicitly* graph-aware!
- **Two kinds of learnable params are jointly learned:**
  - Shallow user/item embeddings
  - GNN's parameters



Shallow user/item embeddings (learnable)

Initial Node Embeddings

- Set the shallow learnable embeddings as the initial node features:
  - For every user $u \in \boldsymbol{U}$, set $\boldsymbol{h}_u^{(0)}$ as the user's shallow embedding.
  - For every item $v \in \boldsymbol{V}$, set $\boldsymbol{h}_v^{(0)}$ as the item's shallow embedding.



Learnable shallow user/item embeddings

Neighbor Aggregation

- Iteratively update node embeddings using neighboring embeddings.

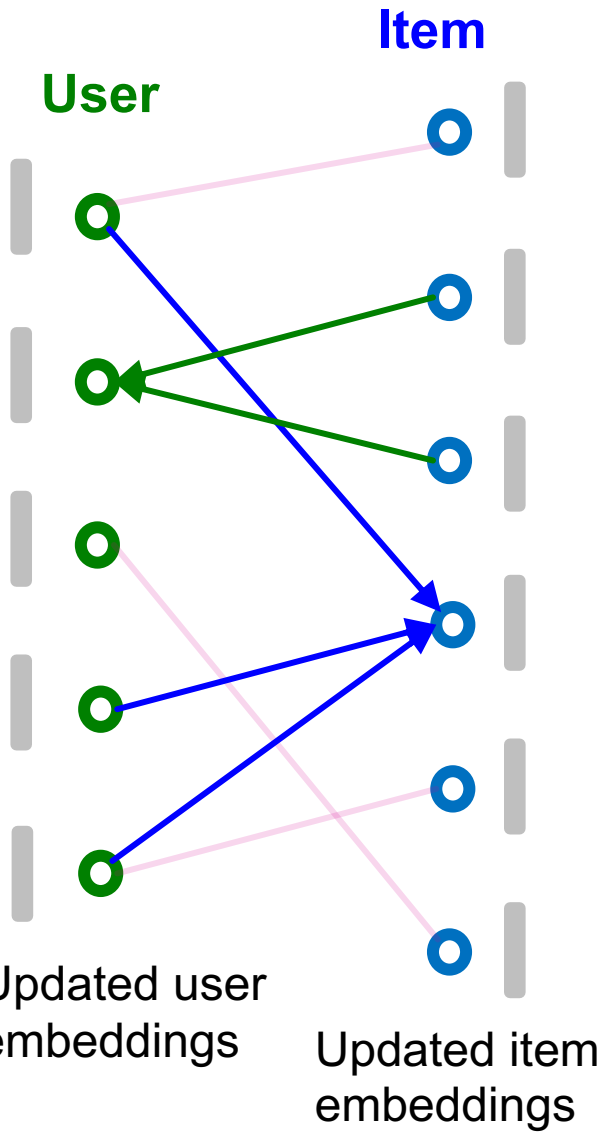
$$\boldsymbol{h}_v^{(k+1)} = \text{COMBINE}\left(\boldsymbol{h}_v^{(k)}, \text{AGGR}\left(\left\{\boldsymbol{h}_u^{(k)}\right\}_{u \in N(v)}\right)\right)$$

$$\boldsymbol{h}_u^{(k+1)} = \text{COMBINE}\left(\boldsymbol{h}_u^{(k)}, \text{AGGR}\left(\left\{\boldsymbol{h}_v^{(k)}\right\}_{v \in N(u)}\right)\right)$$

High-order graph structure is captured through iterative neighbor aggregation.

Different architecture choices are possible for AGGR and COMBINE.

- $\text{AGGR}(\cdot)$ can be $\text{MEAN}(\cdot)$
- $\text{COMBINE}(\boldsymbol{x}, \boldsymbol{y})$ can be $\text{ReLU}(\text{Linear}(\text{Concat}(\boldsymbol{x}, \boldsymbol{y})))$



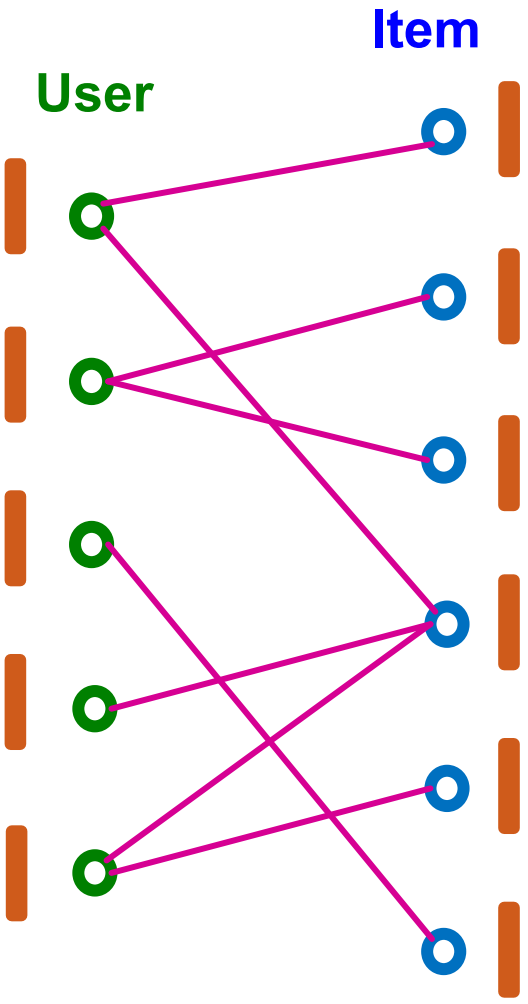
Final Embeddings and Score Function

- After K rounds of neighbor aggregation, we get the **final user/item embeddings** $\boldsymbol{h}_u^{(K)}$ and $\boldsymbol{h}_v^{(K)}$.
- For all $u \in \boldsymbol{U}, v \in \boldsymbol{V}$, we set

$$\boldsymbol{u} \leftarrow \boldsymbol{h}_u^{(K)}, \boldsymbol{v} \leftarrow \boldsymbol{h}_v^{(K)}.$$

- Score function is the inner product

$$\text{score}(u, v) = \boldsymbol{u}^T \boldsymbol{v}$$



Final user/item embeddings (graph-aware)

NGCF: Summary

- Conventional collaborative filtering uses shallow user/item embeddings.
  - The embeddings do ***not explicitly*** **model graph structure**.
  - The training objective **does not model high-order graph structure.**
- **NGCF uses a GNN to propagate the shallow embeddings.**
  - The embeddings are **explicitly aware of high-order graph structure.**

Stanford CS224W: LightGCN

CS224W: Machine Learning with Graphs
Jure Leskovec, Stanford University
http://cs224w.stanford.edu



LightGCN: Motivation (1)

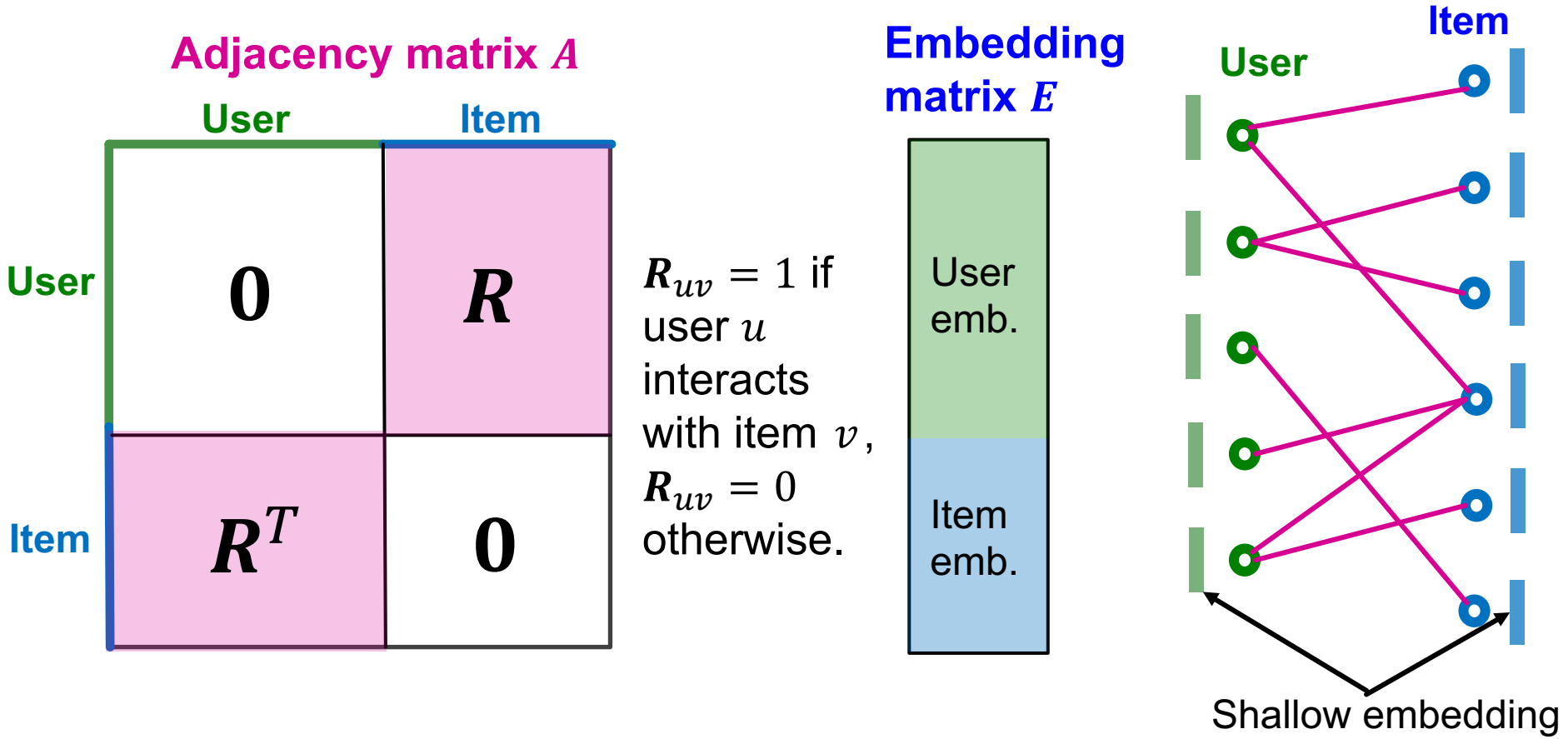
- **Recall**: NGCF jointly learns two kinds of parameters:
  - Shallow user/item embeddings
  - GNN's parameters
- **Observation**: Shallow learnable embeddings are already quite expressive.
  - They are learned for every (user/item) node.
  - Most of the parameter counts are in shallow embeddings when N (#nodes) $\gg$ D (embedding dimensionality)
    - Shallow embeddings: $O(ND)$.
    - GNN: $O(D^2)$.
  - The GNN parameters may not be so essential for performance.

LightGCN: Motivation (2)

- Can we simplify the GNN used in NGCF (e.g., remove its learnable parameters)?
  - **Answer**: Yes!
  - **Bonus**: Simplification improves the recommendation performance!
- **Overview of the idea:**
  - Adjacency matrix for a bipartite graph
  - Matrix formulation of GCN
  - Simplification of GCN by removing non-linearity
    - Related: SGC for scalable GNN [Wu et al. 2019]

Adjacency and Embedding Matrices

- **Adjacency matrix** of a (undirected) bipartite graph.
- **Shallow embedding matrix**.



Matrix Formulation of GCN

- **Define: The diffusion matrix**
- Let $\boldsymbol{D}$ be the degree matrix of $\boldsymbol{A}$.
- Define the normalized adjacency matrix $\tilde{\boldsymbol{A}}$ as

$$\tilde{\boldsymbol{A}} \equiv \boldsymbol{D}^{-1/2}\boldsymbol{A}\boldsymbol{D}^{-1/2}$$

Note: Different from the original GCN, self-connection is omitted here.

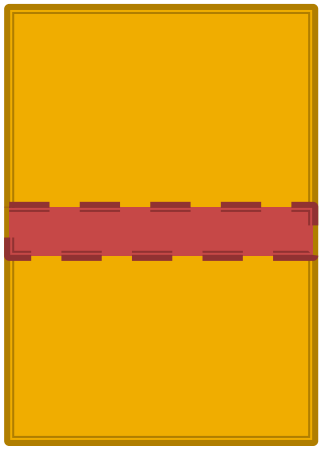
- Let $\boldsymbol{E}^{(k)}$ be the embedding matrix at k-th layer.
- Each layer of GCN's aggregation can be written in a matrix form:

$$\boldsymbol{E}^{(k+1)} = \mathrm{ReLU}\left(\tilde{\boldsymbol{A}}\boldsymbol{E}^{(k)}\boldsymbol{W}^{(k)}\right)$$

Neighbor aggregation

Learnable linear transformation

Matrix of node embeddings $\boldsymbol{E}^{(k)}$



Each row stores node embedding

Simplifying GCN (1)

- **Simplify GCN by removing ReLU non-linearity:**

$$\boldsymbol{E}^{(k+1)} = \tilde{\boldsymbol{A}}\boldsymbol{E}^{(k)}\boldsymbol{W}^{(k)}$$

Original idea from SGC [Wu et al. 2019]

- The final node embedding matrix is given as

$$\boldsymbol{E}^{(K)} = \tilde{\boldsymbol{A}}\,\boldsymbol{E}^{(K-1)}\boldsymbol{W}^{(K-1)}$$

$$= \tilde{\boldsymbol{A}}\left(\tilde{\boldsymbol{A}}\boldsymbol{E}^{(K-2)}\boldsymbol{W}^{(K-2)}\right)\boldsymbol{W}^{(K-1)}$$

$$= \tilde{\boldsymbol{A}}\left(\tilde{\boldsymbol{A}}\left(\cdots\left(\tilde{\boldsymbol{A}}\boldsymbol{E}^{(0)}\boldsymbol{W}^{(0)}\right)\cdots\right)\boldsymbol{W}^{(K-2)}\right)\boldsymbol{W}^{(K-1)}$$

Set $\boldsymbol{E}$ as input embedding $\boldsymbol{E}^{(0)}$

$$= \tilde{\boldsymbol{A}}^{K}\,\boldsymbol{E}\left(\boldsymbol{W}^{(0)}\cdots\boldsymbol{W}^{(K-1)}\right)$$

Simplifying GCN (2)

- Removing ReLU significantly simplifies GCN!

$$\boldsymbol{E}^{(K)} = \boxed{\widetilde{\boldsymbol{A}}^{K}\ \boldsymbol{E}}\ \boldsymbol{W}$$

$$\boldsymbol{W} \equiv \boldsymbol{W}^{(0)} \ldots \boldsymbol{W}^{(K-1)}$$

Diffusing node embeddings along the graph

- **Algorithm**: Apply $\boldsymbol{E} \leftarrow \widetilde{\boldsymbol{A}}\ \boldsymbol{E}$ for K times.
  - Each matrix multiplication diffuses the current embeddings to their one-hop neighbors.
  - **Note:** $\widetilde{\boldsymbol{A}}^{K}$ is dense and never gets materialized. Instead, the above iterative matrix-vector product is used to compute $\widetilde{\boldsymbol{A}}^{K}\ \boldsymbol{E}$.

Multi-Scale Diffusion

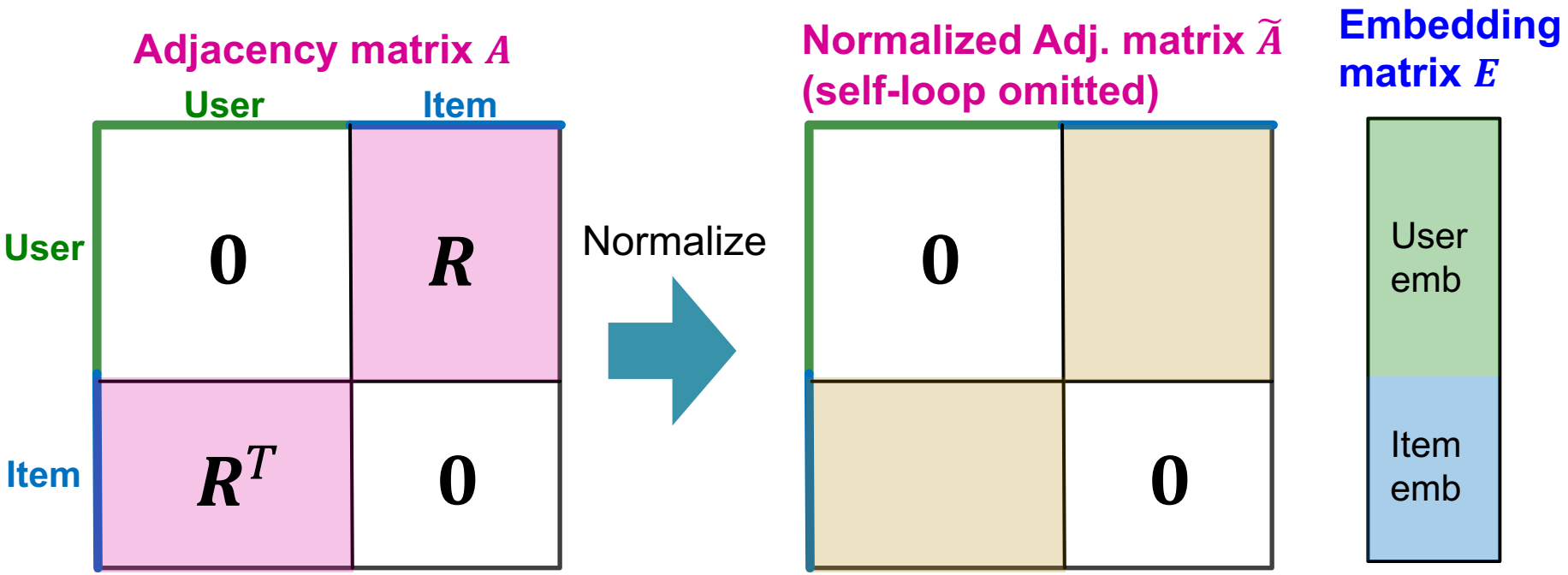
- We can consider **multi-scale diffusion**

$$\alpha_0 E^{(0)} + \alpha_1 E^{(1)} + \alpha_2 E^{(2)} + \cdots + \alpha_K E^{(K)}$$

  - The above includes embeddings diffused at multiple hop scales.
  - $\alpha_0 E^{(0)} = \alpha_0 \tilde{\boldsymbol{A}}^0 E^{(0)}$ acts as a self-connection (that is omitted in the definition $\tilde{\boldsymbol{A}}$)
  - The coefficients, $\alpha_0, \ldots, \alpha_K$, are hyper-parameters.
- For simplicity, LightGCN uses the uniform coefficient, i.e., $\alpha_k = \frac{1}{K+1}$ for $k = 0, \ldots, K$.

LightGCN: Model Overview (1)

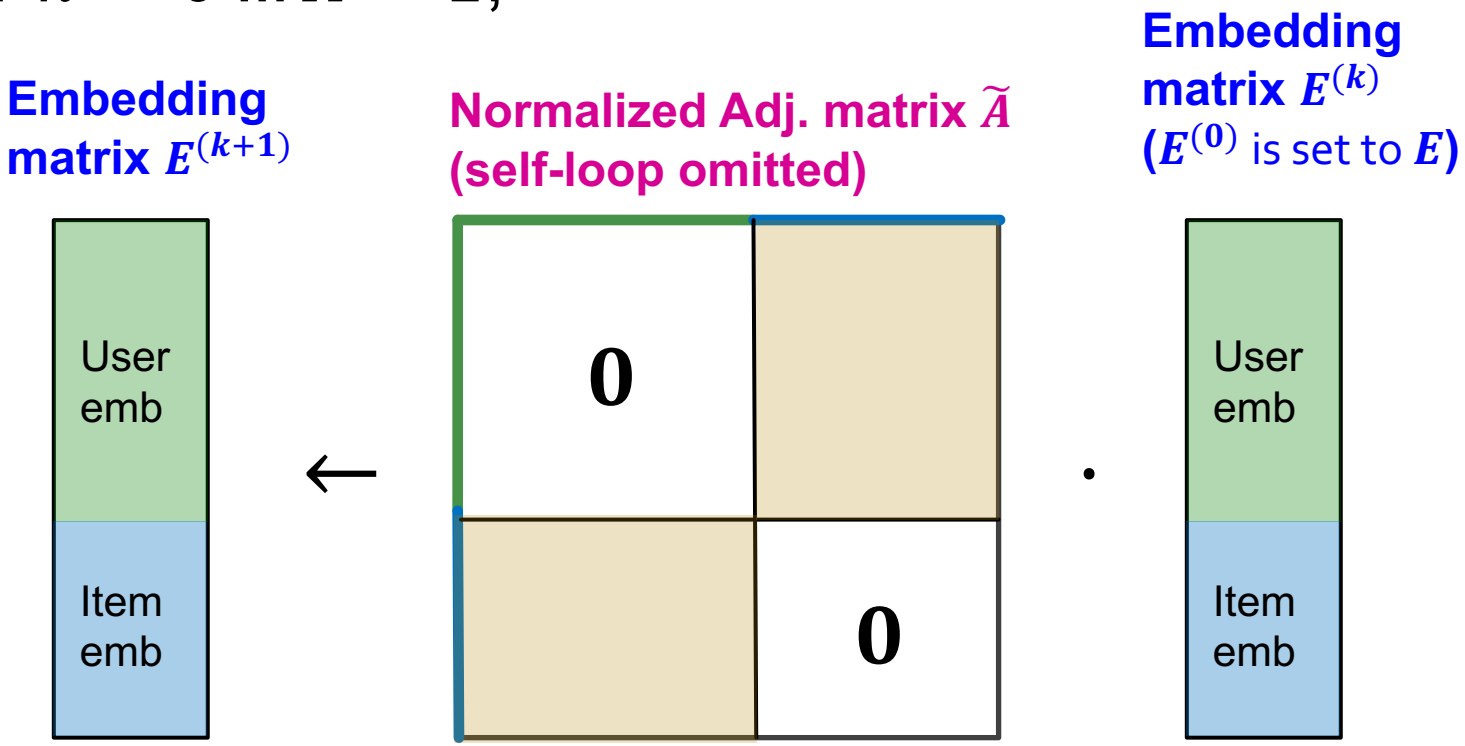
- **Given:**
  - **Adjacency matrix A**
  - **Initial learnable embedding matrix E**



LightGCN: Model Overview (2)

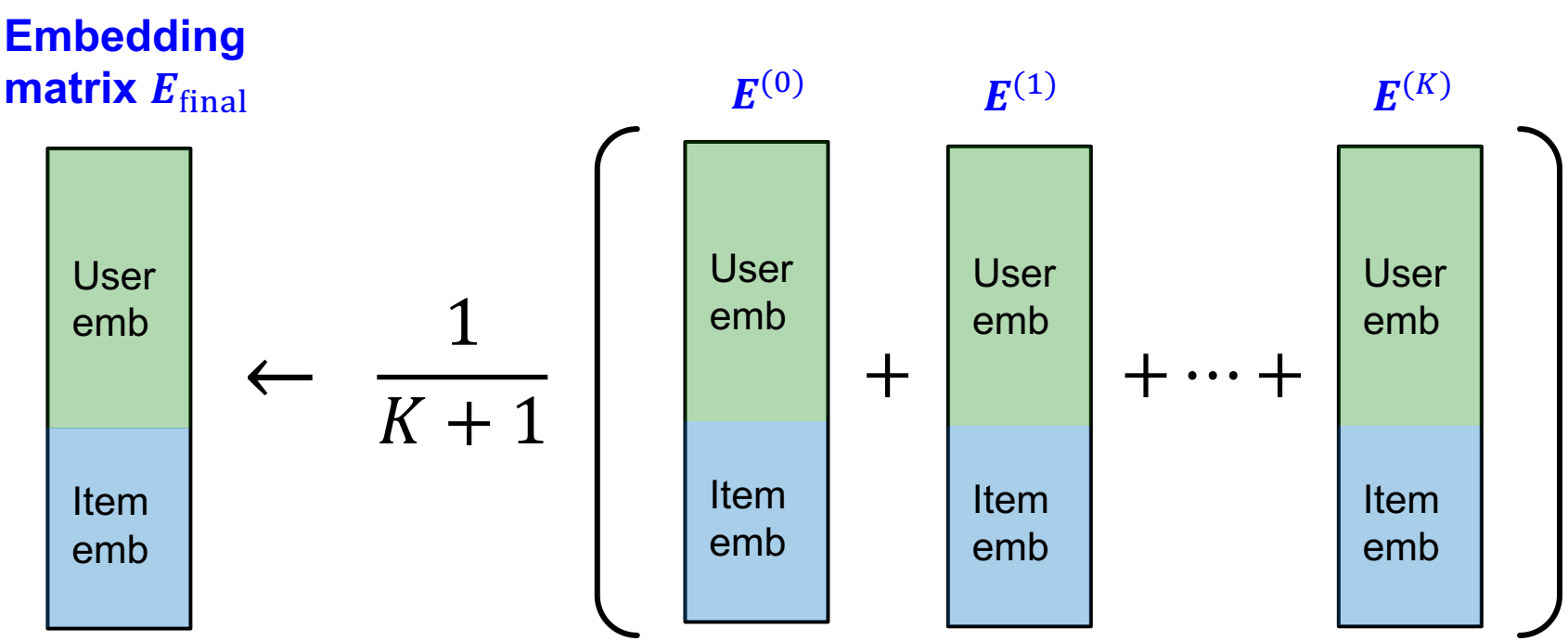
Iteratively diffuse embedding matrix $\boldsymbol{E}$ using $\widetilde{\boldsymbol{A}}$

For $k = 0 \ldots K-1$,



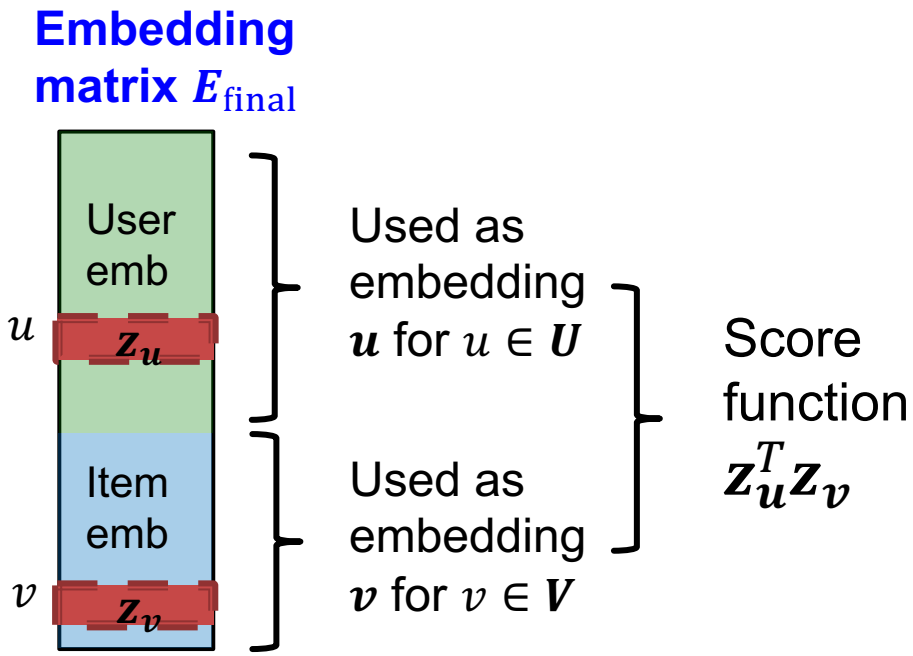
LightGCN: Model Overview (3)

- Average the embedding matrices at different scales.



LightGCN: Model Overview (4)

- **Score function**:
  - Use user/item vectors from $\boldsymbol{E}_{\text{final}}$ to score user-item interaction



LightGCN: Intuition

- **Question**: **Why does the simple diffusion propagation work well?**
- **Answer**: The diffusion directly encourages the embeddings of similar users/items to be similar.
  - Similar users share many common neighbors (items) and are expected to have similar future preferences (interact with similar items).

LightGCN and GCN

- The embedding propagation of LightGCN is closely related to GCN
- **Recall**: GCN (neighbor aggregation part)

$$\boldsymbol{h}_v^{(k+1)} = \sum_{u \in N(v)} \frac{1}{\sqrt{d_u}\sqrt{d_v}} \cdot \boldsymbol{h}_u^{(k)}$$

Node degree

  - Self-loop is added in the neighborhood definition.
- LightGCN uses the same equation except that
  - Self-loop is *not* added in the neighborhood definition.
  - Final embedding takes the average of embeddings from all the layers: $\boldsymbol{h}_v = \frac{1}{K+1}\sum_{k=0}^{K} \boldsymbol{h}_v^{(k)}$.

LightGCN and MF: Comparison

- Both LightGCN and shallow encoders **learn a unique embedding for each user/item.**
- The difference is that LightGCN uses the ***diffused*** user/item embeddings for scoring.
- LightGCN performs better than shallow encoders but are also more computationally expensive due to the additional diffusion step.
  - The final embedding of a user/item is obtained by aggregating embeddings of its multi-hop neighboring nodes.

LightGCN: Summary

- LightGCN simplifies NGCF by **removing the learnable parameters of GNNs.**
- **Learnable parameters are all in the shallow input node embeddings.**
  - Diffusion propagation only involves matrix-vector multiplication.
  - The simplification leads to better empirical performance than NGCF.

Stanford CS224W: PinSAGE

CS224W: Machine Learning with Graphs

Jure Leskovec, Stanford University

http://cs224w.stanford.edu

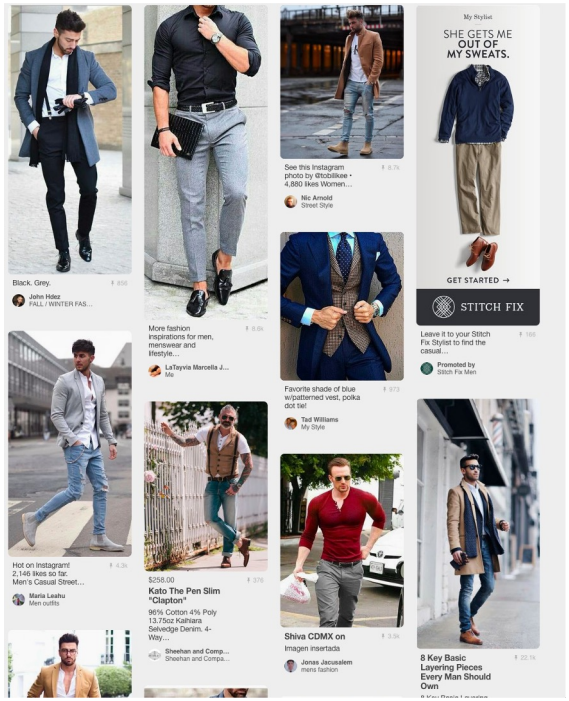


Motivation

- P2P recommendation







PinSAGE: Pin Embedding

- Unifies visual, textual, and graph information.
- The largest industry deployment of a Graph Convolutional Networks.
- Huge Adoption across Pinterest
- Works for fresh content and is available in a few seconds after pin creation



Graph Convolutional Neural Networks for Web-Scale Recommender Systems, Ying et al., 2018

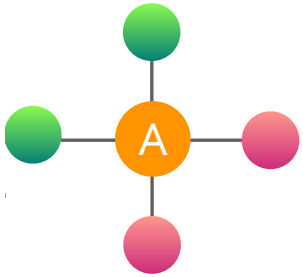
Application: Pinterest

PinSage graph convolutional network:

- **Goal:** Generate embeddings for nodes in a large-scale Pinterest graph containing billions of objects
- **Key Idea:** Borrow information from nearby nodes
  - E.g., bed rail Pin might look like a garden fence, but gates and beds are rarely adjacent in the graph







  - Pin embeddings are essential to various tasks like recommendation of Pins, classification, ranking
    - Services like “Related Pins”, “Search”, “Shopping”, “Ads”

Harnessing Pins and Boards



Very ape blue structured coat

Nitty Gritty

Picked for you
Street style



Hans Wegner chair

Room and Board

Promoted by
Room & Board



This is just a beautiful image for thoughts.
Yay or nay, your choice.

14

Annie Teng
Plantation



mid century modern ...
MJL I -



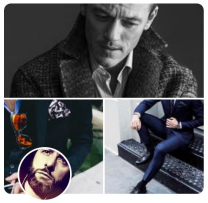
Man Style
Gavin Jones



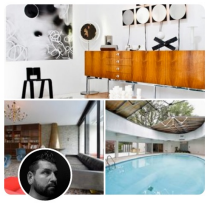
men + style I
F I G + S A L T



Plants
HelloSandwich



Men's Style
Andrea Sempi



Mid century modern
Tyler Goodro



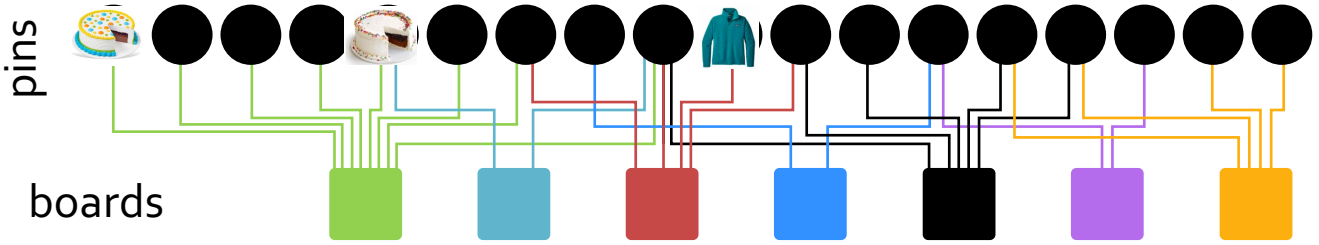
Plants
Moorea Seal

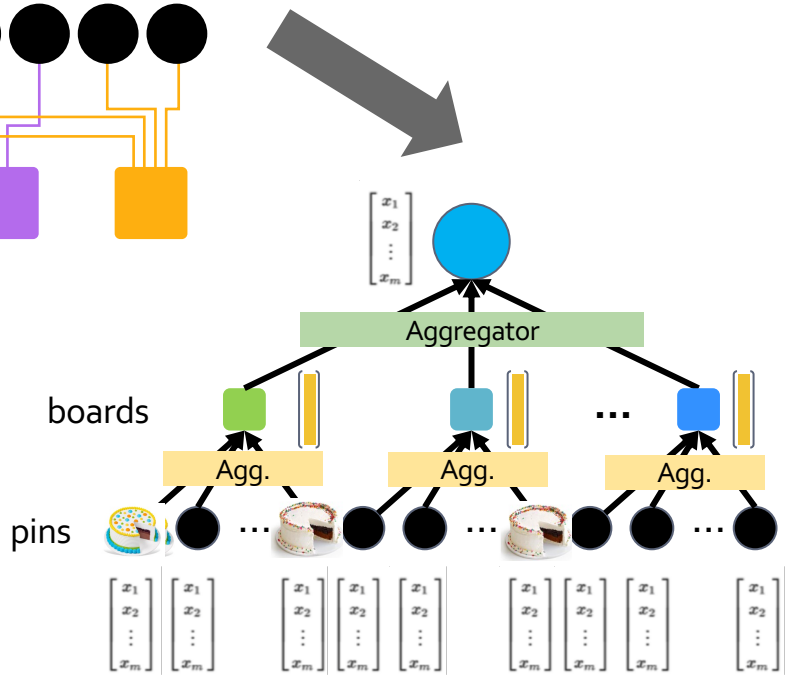


Mid century modern ...
Prettygreentea

PinSAGE:Graph Neural Network

- Graph has tens of billions of nodes and edges
- Further resolves embeddings across the Pinterest graph

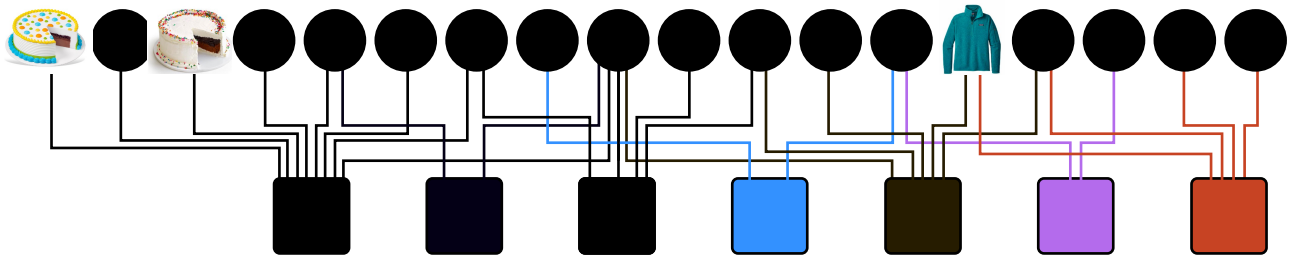




PinSAGE: Methods for Scaling-Up

- In addition to the GNN model, the PinSAGE introduces several methods to scale the GNN to a billion-scale recommender system (e.g., Pinterest).
  - Shared negative samples across users in a mini-batch
  - Hard negative samples
  - Curriculum learning
  - Mini-batch training of GNNs on a large-graph (to be covered in the future lecture)

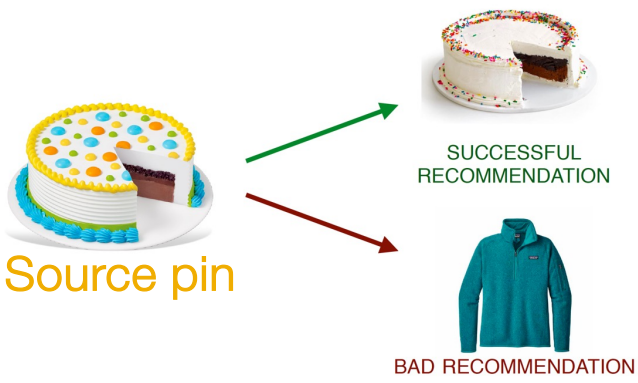
PinSAGE Model

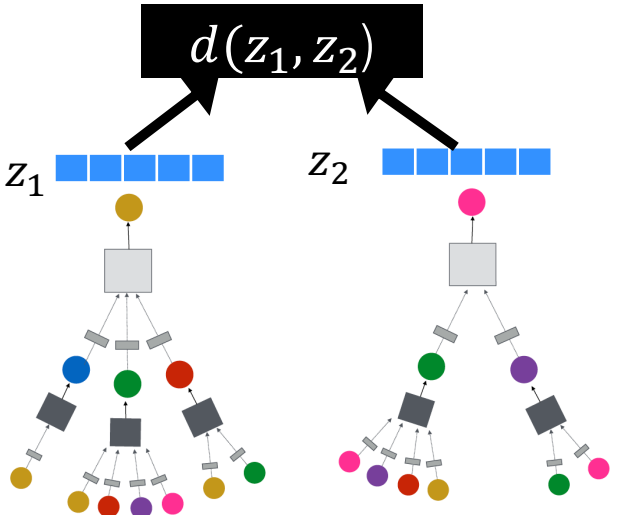


Task: Recommend related pins to users

Learn node embeddings z_i such that

$$d(z_{cake1}, z_{cake2}) < d(z_{cake1}, z_{sweater})$$





Training Data

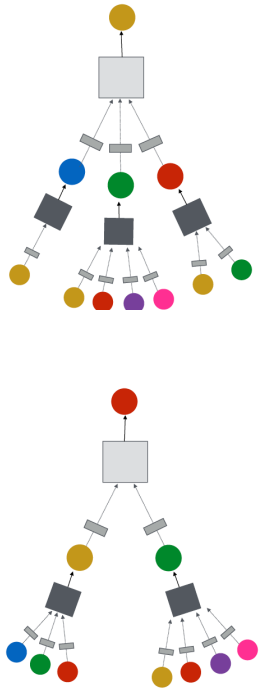
- 1+B repin pairs:
  - From Related Pins surface
  - Capture semantic relatedness
  - Goal: Embed such pairs to be “neighbors”

- Example positive training pairs (Q,X):



Shared Negative Samples (1)

- **Recall**: In BPR loss, for each user $u^* \in \boldsymbol{U}_{\text{mini}}$, we sample one positive item v_{pos} and a set of sampled negative items $\boldsymbol{V}_{\text{neg}} = \{v_{\text{neg}}\}$.
- Using more negative samples per user improves the recommendation performance, but is also expensive.
  - We need to generate $|\boldsymbol{U}_{\text{mini}}| \cdot |\boldsymbol{V}_{\text{neg}}|$ embeddings for negative nodes.
  - We need to apply $|\boldsymbol{U}_{\text{mini}}| \cdot |\boldsymbol{V}_{\text{neg}}|$ GNN computational graphs (see right), which is expensive.



Shared Negative Samples (2)

- **Key idea**: We can share the same set of negative samples $\boldsymbol{V}_{\text{neg}} = \{v_{\text{neg}}\}$ ***across all users*** $\boldsymbol{U}_{\text{mini}}$ in the mini-batch.
- This way, we only need to generate $|\boldsymbol{V}_{\text{neg}}|$ embeddings for negative nodes.
  - This saves the node embedding generation computation **by a factor of** $|\boldsymbol{U}_{\text{mini}}|$!
  - Empirically, the performance stays similar to the non-shared negative sampling scheme.

Hard Negatives (1)

- **Challenge**: Industrial recsys needs to make **extremely fine-grained predictions**.
  - #Total items: Up to billions.
  - #Items to recommend for each user: 10 to 100.
- **Issue**: The shared negative items are randomly sampled from all items
  - Most of them are "**easy negatives**", i.e., a model does not need to be fine-grained to distinguish them from positive items.
- We need a way to sample "**hard negatives**" to force the model to be fine-grained!

PinSAGE: Curriculum Learning

- **Idea:** use harder and harder negative samples
- Include more and more hard negative samples for each epoch

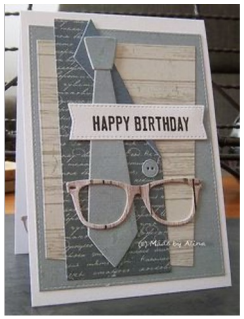


Source pin



Positive


Easy negative



Hard negative

Curriculum Learning

- **Key insight**: It is effective **to make the negative samples *gradually harder* in the process of training**.
- At n-th epoch, we add $n-1$ hard negative items.
  - #(Hard negatives) gradually increases in the process of training.
- The model will gradually learn to make finer-grained predictions.

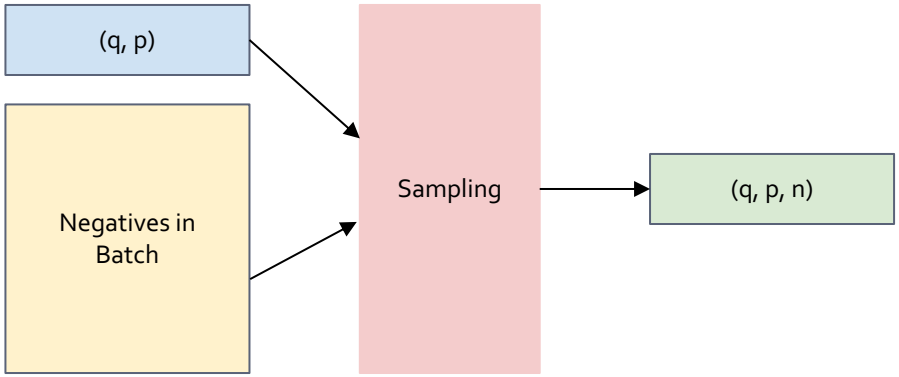
Hard Negatives (2)

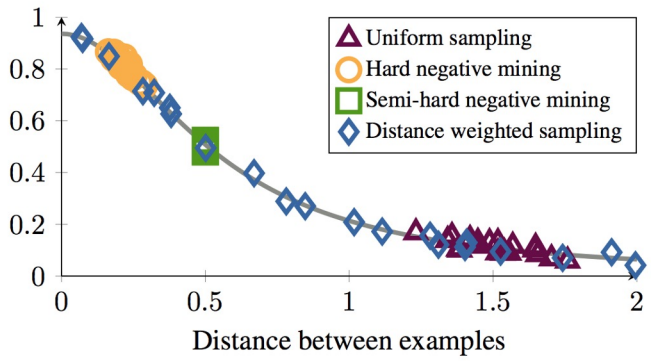
- **For each user node**, the **hard negatives** are item nodes that are close (but not connected) to the user node in the graph.
- Hard negatives for user $u \in \boldsymbol{U}$ are obtained as follows:
  - Compute random walks from user u.
    - Run random walk with restart from u, obtain visit counts for other items/nodes.
  - Sort items in the descending order of their visit count.
  - Randomly sample items that are ranked high but not too high, e.g., 2000th —5000th .
    - Item nodes that are close but not too close (connected) to the user node.
- The hard negatives for each user are used in addition to the shared negatives.

PinSAGE: Negative Sampling

(q, p) positive pairs are given but various methods to sample negatives to form (q, p, n)

- Distance Weighted Sampling (Wu et al., 2017)
  - Sample negatives so that query-negative distance distribution is approx uniform U[0.5, 1.4]





(b) Sample distribution for different strategies.

Fine-Grained Object Similarity

Query



Visual only



PinSAGE

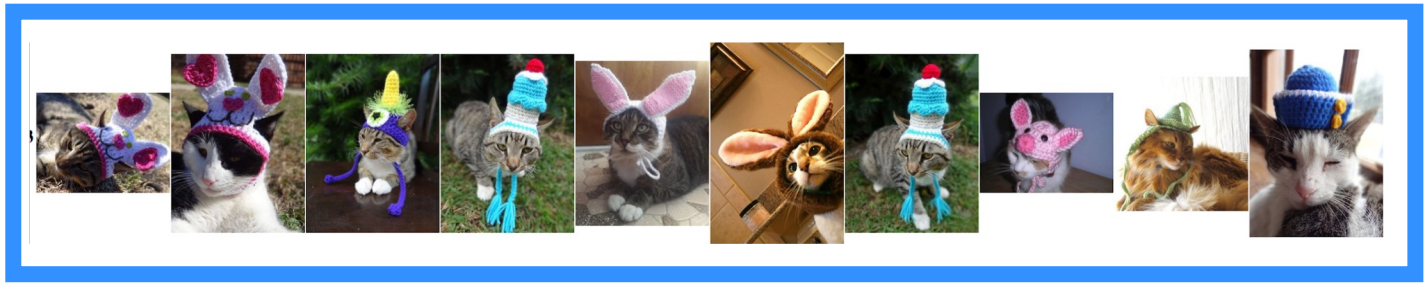


Compare against Prod

Query







PinSAGE: Summary

- **PinSAGE uses GNNs** to generate high-quality user/item embeddings that **capture both the rich node attributes and graph structure**.
- The PinSAGE model is effectively trained using sophisticated **negative sampling strategies**.
- PinSAGE is **successfully deployed at Pinterest**, a billion-scale image content recommendation service.